



Today's agenda

↳ Trie basics.

↳ check if given word exists.



AlgoPrep



Q) Given N strings and Q queries. For each query check if given query is in given N strings.

$N=8$

$Q=3$

//idea1

damp

drake

↳ for every query, iterate on the input strings and match them.

dark

data

T.C: $Q * (N * L)$ S.C: $O(L)$

data

damp xx

drake

draw

Cash

//idea2

Crew

↳ store all the input strings in Hashset/HashMap. for every query search in hashset.

mango

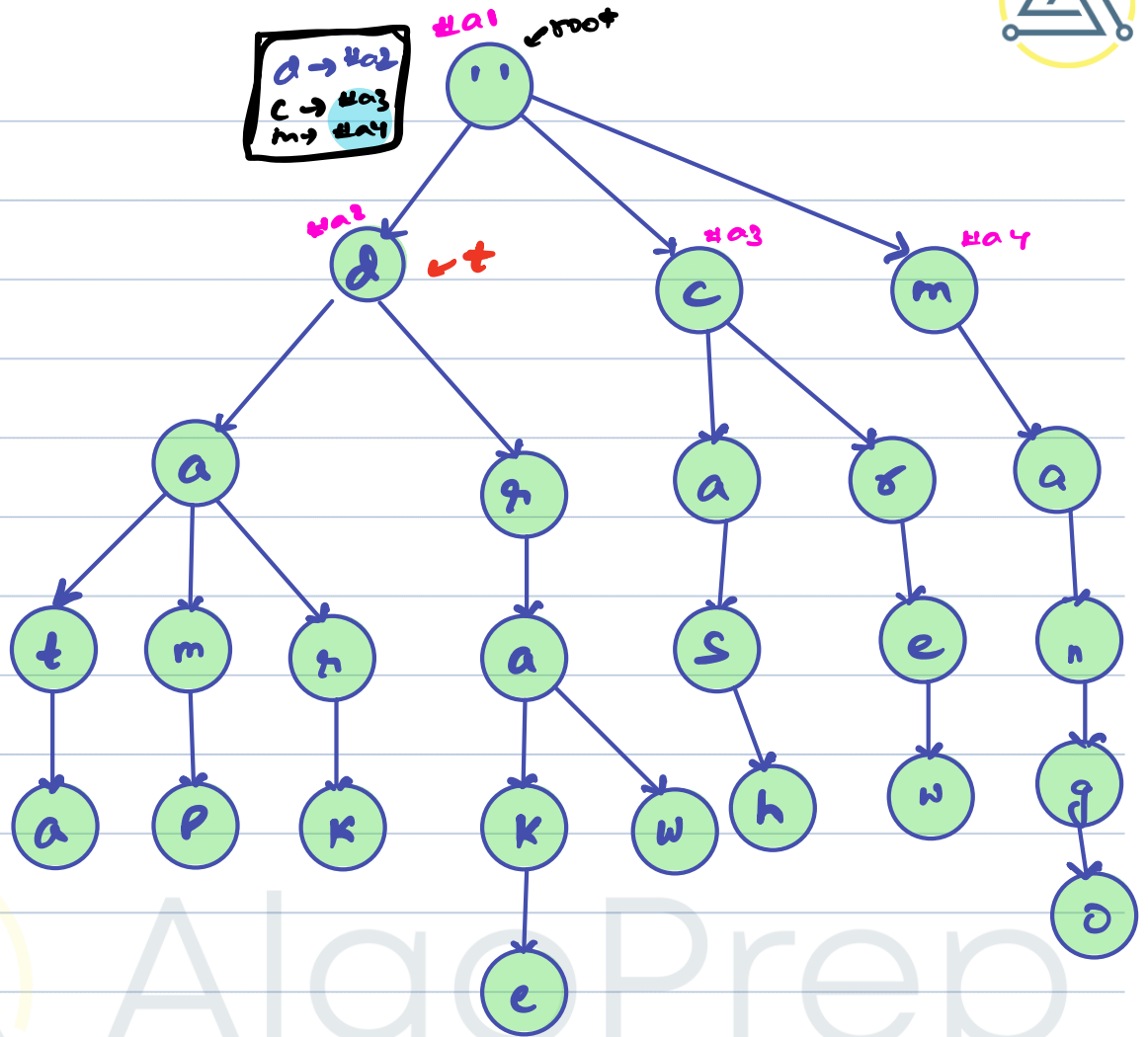
T.C: $O(N+Q)$ S.C: $O(N)$

```
class Node {  
    int val;  
    Node left;  
    Node right;  
}
```



Idea 3

N=8
damp
dark
data
drake
draw
dash
crew
→ mango



Node {

~~char data;~~

HashMap < Char, Node > hm;

boolean row;

}



// Pseudo code

```
void insert (Node root, String data){  
    Node t = root;  
  
    for (int i=0; i<data.length(); i++){  
        char ch = data.charAt(i);  
  
        if (t.hm.containsKey(ch) == true){  
            t = t.hm.get(ch);  
        }  
        else{  
            Node n1 = new Node();  
            n1.data = ch;  
            t.hm.put(ch, n1);  
            t = t.hm.get(ch);  
        }  
    }  
    t.low = true;  
}
```

$O(N+1)$

Tracing

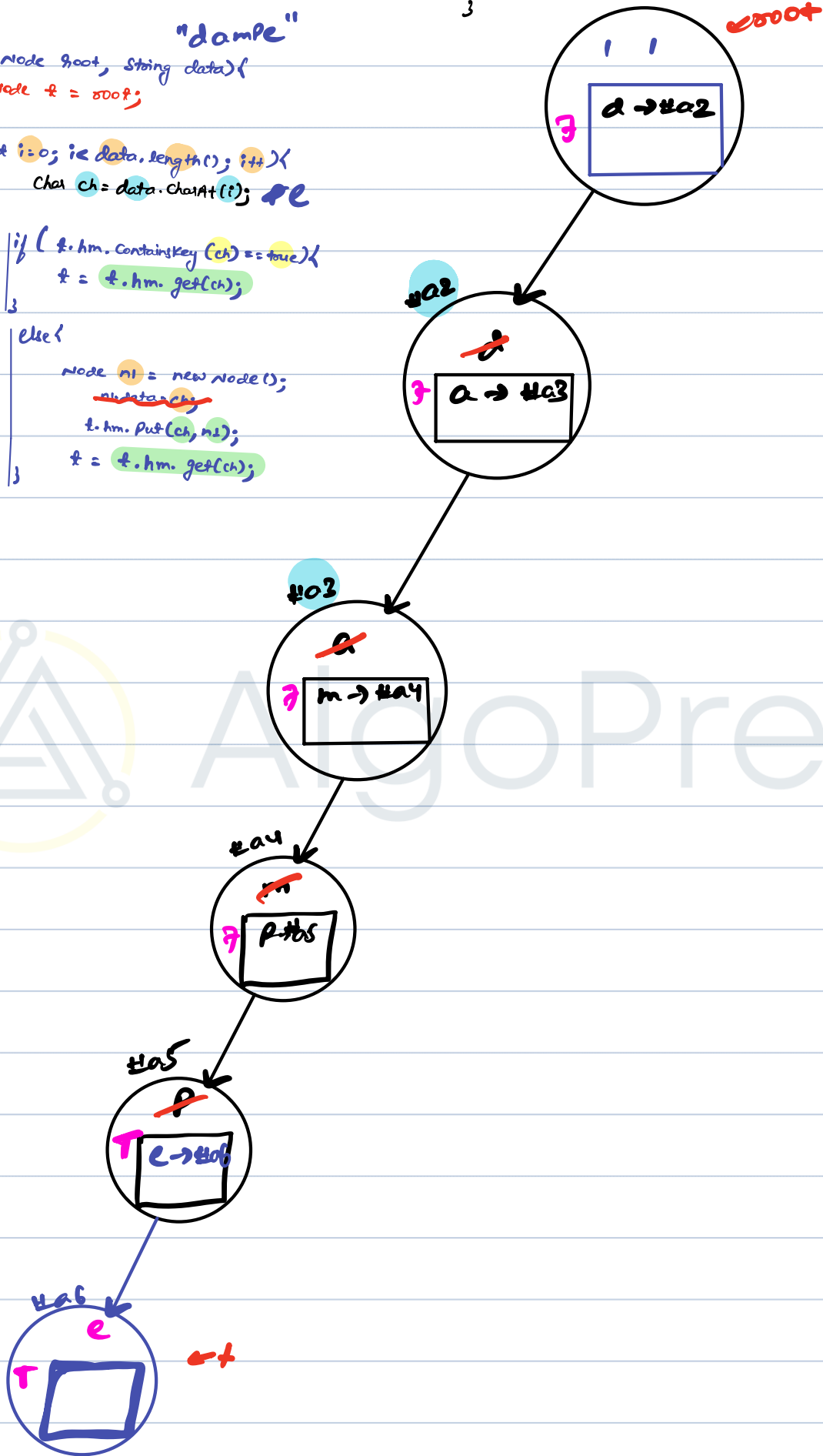
"dame"

```
void insert(Node root, String data){
    Node t = root;
```

```
    for (int i=0; i<data.length(); i++){
        char ch = data.charAt(i);

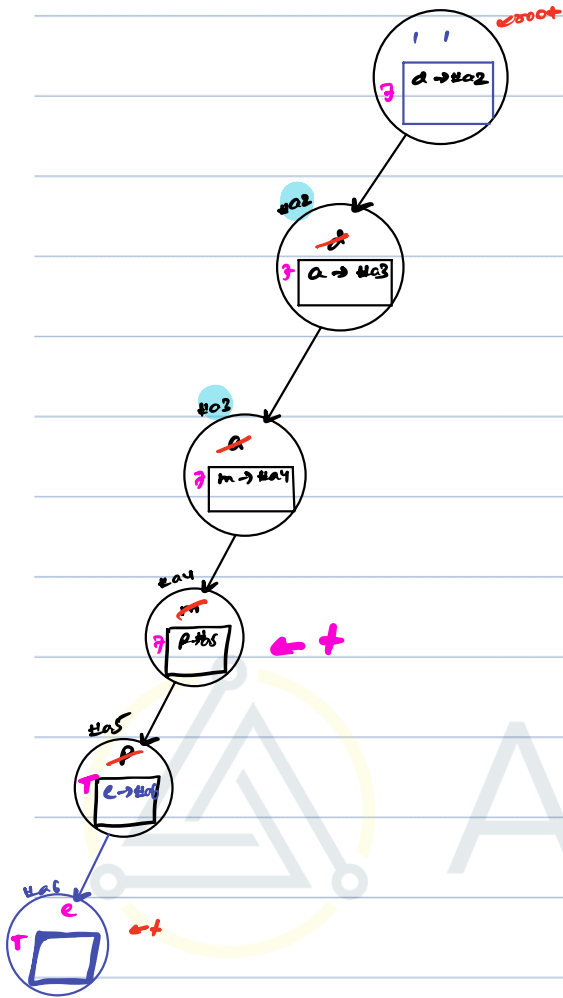
        if (!t.hm.containsKey(ch)){
            t = t.hm.get(ch);
        }
        else{
            Node n1 = new Node();
            n1.data = ch;
            t.hm.put(ch, n1);
            t = t.hm.get(ch);
        }
    }
}
```

```
Node {
    char data;
    HashMap < char, Node > hm;
}
```





// Query Solving



→ dam

```
boolean search(Node root, String data){  
    Node t = root;
```

```
    for(int i=0; i<data.length(); i++){  
        char ch = data.charAt(i);
```

```
        if(t.hm.containsKey(ch) == true){  
            t = t.hm.get(ch);
```

```
        }  
        else{
```

```
            return false;
```

```
        }
```

```
    }  
    if(t.isEnd == true){ return true; }  
    else { return false; }
```

```
}
```



```
boolean search (Node root, String data) {
```

```
    Node t = root;
```

```
    for (int i=0; i<data.length(); i++) {
```

```
        char ch = data.charAt(i);
```

```
        if (t.hm.containsKey(ch) == true) {
```

```
            t = t.hm.get(ch);
```

```
        }
```

```
        else {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    return t.ew;
```

```
}
```

$O(n)$

sc: $O(n \times k)$



Q) Given N strings and Q queries, Calculate number of Prefix strings of given words.

$N=8$

$Q=3$

//idea1

damp

da $\rightarrow 3$

dark

dr $\rightarrow 2$

data

ca $\rightarrow 2$

drake

draw

cash

Crew

Cashify

↳ for every query, iterate on the input strings and match them.

T.C: $Q * (N * L)$ S.C: $O(L)$

//idea2

↳ Store all the input strings in Hashset/HashMap for every query search in Hashset.

T.C: $O(N+Q)$ S.C: $O(N)$

Node {

int PC;

HashMap < char, Node > hm;

boolean row;

}



$N=8$ $cl=3$

damp $da \rightarrow 3$

dark

data

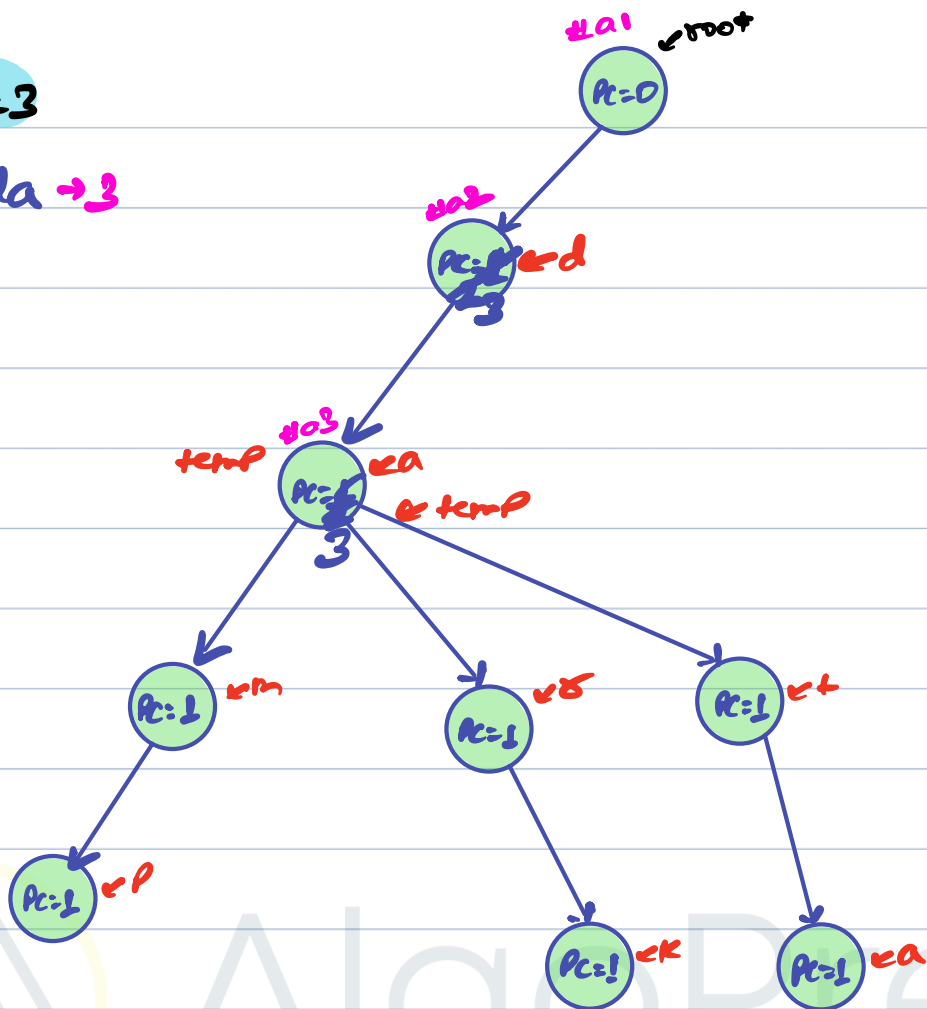
drake

draw

dash

draw

dashy



AlgoPrep



// Pseudo code

```
void insert (Node root, String data){  
    Node t = root;  
  
    for (int i=0; i<data.length(); i++){  
        char ch = data.charAt(i);  
  
        if (t.hm.containsKey(ch) == true){  
            t = t.hm.get(ch);  
            t.pc++;  
        }  
        else{  
            Node n1 = new Node();  
            n1.data = ch;  
            t.hm.put(ch, n1);  
            t = t.hm.get(ch);  
            t.pc++;  
        }  
    }  
    t.low = true;  
}
```

$O(N+1)$



```
int search (Node root, String data) {
```

```
    Node t = root;
```

```
    for (int i=0; i<data.length(); i++) {  
        char ch = data.charAt(i);
```

```
        if (t.hm.containsKey(ch) == true) {  
            t = t.hm.get(ch);
```

```
        }
```

```
        else {
```

```
            return 0;
```

```
        }
```

```
    }  
  
    return t.pc;
```

```
}
```

$O(n)$

String + Prefix $\rightarrow$ Trie